University of **P**aris-**E**st **C**réteil (UPEC)
École **P**ublique d'**I**ngénieurs de la **S**anté **E**t du **N**umérique (EPISEN)
Saint-Gobain GDII

How can we detect cheating effectively without compromising system security through kernel-level access?

Presented by Gilles MEUNIER

gilles.meunier@etu.u-pec.fr

May 2026

Jury:

Olivier MICHEL	Tutor
Hervé PERRACHON	Mentor
James ORTIZ	Teacher
Surname NAME	Teacher

Acknowledgements

I thank my friends for introducing me to Counter-Strike.

Contents

List of Figures

Introduction

If you look at a multi-player game, it's the people who are playing the game who are often more valuable than all of the animations and models and game logic that's associated with it.

– Gabe NEWELL

Context

Online competitive video games, particularly First-Person Shooters (FPS), now occupy a central place in both the video game industry and the esports ecosystem. Their model relies on technical fairness between players: each participant must be subject to the same rules, mechanical constraints, and informational limitations.

This is nowhere more visible than in Counter-Strike 2 (CS2), Valve's flagship tactical shooter and one of the most-watched titles in professional esports. The Counter-Strike series in particular has accumulated a continuous record of adversarial development between cheats and anti-cheat systems since the first VAC (VAC) release in 2002, with each new engine generation triggering an immediate re-emergence of cheat software within days of launch [1]. Major tournaments distribute cash prizes reaching several million dollars, and top-tier organizations field full-time professional rosters whose careers - and livelihoods - depend on the perceived integrity of every match. This economic stake is precisely why most high-level CS2 tournaments are played in LAN (LAN) conditions: bringing players into a single physical venue, on hardware controlled by tournament organizers, drastically reduces the attack surface available to cheaters compared to online play, where each participant's machine remains outside the organizers' control. The contrast between these two environments - tightly supervised LAN finals versus the millions of anonymous online matchmaking games played daily - illustrates the scale of the challenge: competitive integrity cannot realistically be guaranteed everywhere through physical supervision alone.

However, cheating represents a growing threat to this balance. The emergence of automated tools such as aimbots and wallhacks alters match integrity, undermines competitive credibility, and directly affects player trust as well as the economic viability of affected titles. In a game where a single well-placed shot can decide a round, and a round can decide a series worth a cash prize, even a small number of undetected cheaters in the matchmaking pool is enough to erode confidence in the entire ranking and reward system - both for the casual player grinding toward a higher rank, and for the aspiring

professional whose statistics may be scouted by a team.

To address this issue, many publishers have progressively adopted kernel-level anti-cheat systems. These solutions operate with very high privileges, allowing them to monitor memory, processes, and certain hardware interactions within the operating system in order to detect manipulation of the game client. While often effective against certain forms of software cheating, they also raise significant concerns regarding cybersecurity, privacy protection, and user acceptability. The permanent presence of a third-party privileged component increases the system's attack surface, introduces a critical point of failure and fuels public debate about the proportionality between technical intrusion and its intended objective.

Beyond pure cybersecurity vulnerabilities, kernel-level solutions introduce profound privacy and regulatory dilemmas. Operating at the highest privilege layer (Ring 0) theoretically grants these software components unhindered visibility over the user's entire system, including personal files, background applications, and non-game-related data. This continuous monitoring mechanism challenges traditional notions of informed user consent and clashes directly with modern data protection frameworks, such as the GDPR, which mandate strict principles of data minimization and proportionality. Consequently, this technological choice has sparked a sharp divide within the gaming community: while competitive players often tolerate intrusive measures in the name of competitive integrity, casual users increasingly reject what they perceive as a form of digital surveillance, ultimately threatening game adoption and brand reputation.

While cheating in a video game may appear as a low-stakes issue compared to critical infrastructure or enterprise security breaches, the underlying mechanisms (unauthorized memory access, process injection, behavioral detection, and privilege escalation) are directly transferable to higher-stakes contexts. Studying anti-cheat systems therefore offers a low-risk environment in which to examine broader questions of privileged software design and intrusion detection. This relevance is further amplified by the evolving nature of modern cyberattacks: as AI-driven techniques increasingly automate and diversify attack methods, the specific vectors used to compromise a system change rapidly, yet the resulting behavioral footprint (unauthorized access, data exfiltration, persistence) remains comparatively stable. Understanding how a privileged component can access or conceal sensitive data, even in the constrained context of anti-cheat software, thus provides insight applicable to a much broader class of security challenges.

Problem

In this context, the following question arises: can cheating be effectively detected without compromising system security through kernel-level access? This thesis explores an alternative approach primarily based on server-side behavioral analysis and machine learning techniques. Instead of directly inspecting the player's machine, the idea is to leverage gameplay data itself-match statistics, action sequences, and temporal and mechanical consistency-to identify anomalies characteristic of assisted behavior.

Recent research suggests that analyzing replays and gameplay data in games such as Counter-Strike 2 makes it possible to distinguish legitimate players from suspicious profiles using labeled datasets.

However, these approaches also present limitations: the need for large volumes of data, the risk of false positives, delayed detection, and the continuous adaptation of cheaters. The objective of this thesis is therefore twofold. First, to analyze the advantages and limitations of kernel-level anti-cheats in light of cybersecurity, system stability, and user trust considerations. Second, to design and evaluate a behavioral detection system based on gameplay metrics. A proof of concept will be implemented using replay data or simulated logs for a Counter-Strike-like FPS in order to generate a suspicion score and study the prioritization of human review.

Finally, the thesis will propose an architectural positioning of these approaches within a multi-layer anti-cheat system, combining server-side detection, lightweight client mechanisms, and human validation, with the aim of reconciling effectiveness, security, and player acceptability.

Outline

In the first chapter, we will present the state of the art in cheating and anti-cheat technologies. We will analyze the main categories of cheats - automation tools, information cheats, and advanced hardware-assisted vectors - and examine the anti-cheat mechanisms developed in response, from client-side solutions to kernel-level drivers and data-driven detection methods.

In the second chapter, we will assess the risks and structural limitations of kernel-level anti-cheats. We will analyze the technological arms race they sustain, their inability to address hardware-assisted cheating such as DMA (DMA), and their economic and operational costs. This analysis will be supported by an experimental section using Windows-Kernel-Explorer to illustrate the level of system access a privileged kernel component holds, and to discuss the security and privacy implications that follow.

In the third chapter, we will present the design and implementation of a server-side gameplay analytics pipeline for suspicion scoring. We will describe the telemetry data model, the algorithmic architecture combining rule-based heuristics and an LSTM (LSTM) layer, and the results of benchmarking against simulated player profiles. We will evaluate the system's performance, discuss the high-skill bias problem and its mitigations, and compare this approach against the kernel-level paradigm examined in the previous chapter.

Finally, in the conclusion, we will synthesize our findings and propose a positioning of these complementary approaches within a multi-layered anti-cheat architecture, balancing detection effectiveness, system security, and player trust.

Chapter 1

State of the Art - Cheats and Anti-Cheats

Technology is just a tool. In terms of getting the kids working together and motivating them, the teacher is the most important.

– Bill GATES

1.1 Cheat-Based Exploration of System Architecture

1.1.1 Automation Cheats

Automation cheats represent a critical class of malicious software engineered to augment or entirely substitute a player's mechanical execution within the competitive environment. This category primarily encompasses automated aiming tools (aimbots), automatic firing utilities (), and execution macros or scripts. By targeting the core pillars of first-person shooter (FPS) gameplay—specifically mechanical precision, muscle memory, and cognitive reaction time, these tools disrupt competitive balance by replacing human motor control with software-driven operations.

To achieve this automation, the software must interface directly with either the operating system's input layer or manipulate the inner variables governing the game engine logic. The depth of this interaction generally defines the cheat's sophistication and detection difficulty:

- Input Layer Manipulation: Client-side automation tools often operate by injecting synthetic mouse and keyboard events directly into the operating system's input stream. A typical triggerbot, for example, continuously polls the game client's state or memory addresses to determine when an enemy entity's hitbox intersects with the exact coordinates of the player's crosshair. The moment this condition is validated, the cheat programmatically fires a click event into the input queue, completely bypassing human visual processing and nervous system latency. The latency between the detection of the event and the actual input can be configured to try and pass as a human reaction, though the main point of this process is still to be way faster than the average player.

- Subversion of Game Engine Logic: More advanced automation mechanisms bypass peripheral input emulation altogether, executing modifications directly within the local game engine client memory space. An aimbot reads the real-time spatial positions (three-dimensional coordinates) of all active opponents from the engine's local entity list. It then calculates the precise angular vector offsets required to bridge the gap between the current viewpoint and the target's head hitbox. By overwriting the client's view-angle variables within the engine memory before the next frame rendering or tick packet serialization, the cheat forces perfect accuracy.

Figure 1: https://cs2pulse.com/spray-patterns/ak-47/

This enables frame-perfect recoil compensation and automated target tracking that completely overrides the physical constraints designed by the game developers, as illustrated in Figure 1, which confirms the deterministic nature of the AK-47's spray pattern and the trivial feasibility of scripted compensation.

1.1.2 Information Cheats

Unlike automation tools that directly override a player's physical inputs, information cheats focus on subverting the tactical integrity of tactical shooter ecosystems by breaking down spatial boundaries. Tactical first-person shooters heavily rely on a conceptual "fog of war"—a structural design where information regarding enemy positions, utility placement, and movement is deliberately occluded by spatial geometry to reward map control and sensory awareness. Information cheats, primarily manifested as wallhacks and radarhacks, systematically dismantle this baseline restriction. They do not execute actions for the user but instead grant an absolute cognitive advantage by exposing hidden game data and altering the rendering pipeline.

To achieve this asymmetric visualization, these tools typically exploit two distinct technical vectors within the local client system:

- Rendering Pipeline Exploitation: Wallhacks operate by intercepting or altering the graphics rendering pipeline responsible for drawing the game world onto the player's screen. In a legitimate game loop, the engine utilizes occlusion to ensure that player models hidden behind opaque geometry (such as walls or solid terrain) are not drawn on the screen. Wallhacks circumvent this by hooking into the graphic APIs (APIs) used by the engine. By injecting code that forces the graphics card to ignore occlusion parameters when processing enemy entity models, the cheat renders these entities on top of intervening geometry. This is frequently accompanied by ESP (ESP) boxes, which overlay real-time lines, health bars, and item descriptions directly onto the player's view canvas — a result visually indistinguishable from the engine's own debug rendering mode, as shown in Figure 2, where the output of a native CS2 command forcing occlusion bypass (top) is compared side-by-side with a commercial cheat advertisement (bottom).

- Game Data Exposure: Radarhacks bypass the visual canvas entirely, targeting the underlying spatial data structures residing within the client's volatile memory or network socket streams. Every online game client maintains a local "Entity List" containing the precise three-dimensional coordinate vectors of all active participants, transmitted by the server to ensure local interpolation and hit registration. A radarhack continuously scans the process memory space to extract these spatial vectors or sniffs incoming network packets before they are parsed by the client. This data is then projected onto a secondary web-based interface, an external execution window, or a simulated 2D mini-map overlay. Because this methodology leaves the primary visual interface completely clean and does not alter rendering hooks, it provides the cheater with absolute situational awareness while significantly minimizing visibility to automated anti-cheat screen capture mechanisms or human review systems. This type of cheating is one of the hardest to detect as, except for the direct scanning of the process memory space, the behaviour of such cheaters can often be associated with a good game-sense or instincts, hence being harder to prove as blatant cheating — Figure 3 illustrates this extraction pipeline, showing enemy coordinates pulled from the entity list and projected onto a clean secondary interface that leaves the primary game view entirely unaltered.

Figure 2: Comparison of wallhack demonstration using built-in command "r_aoproxy_show 1" in CS2, which forces the engine to render near entity models regardless of occlusion (top) and an ad from a cheating software provider showing wallhack in action (bottom).

1.1.3 Advanced Cheats

As client-side anti-cheat mechanisms evolved to actively monitor user-mode operations, the cheating ecosystem shifted toward highly sophisticated execution paradigms categorized as advanced cheats. These advanced threats deliberately bypass the operating system's standard application boundaries by

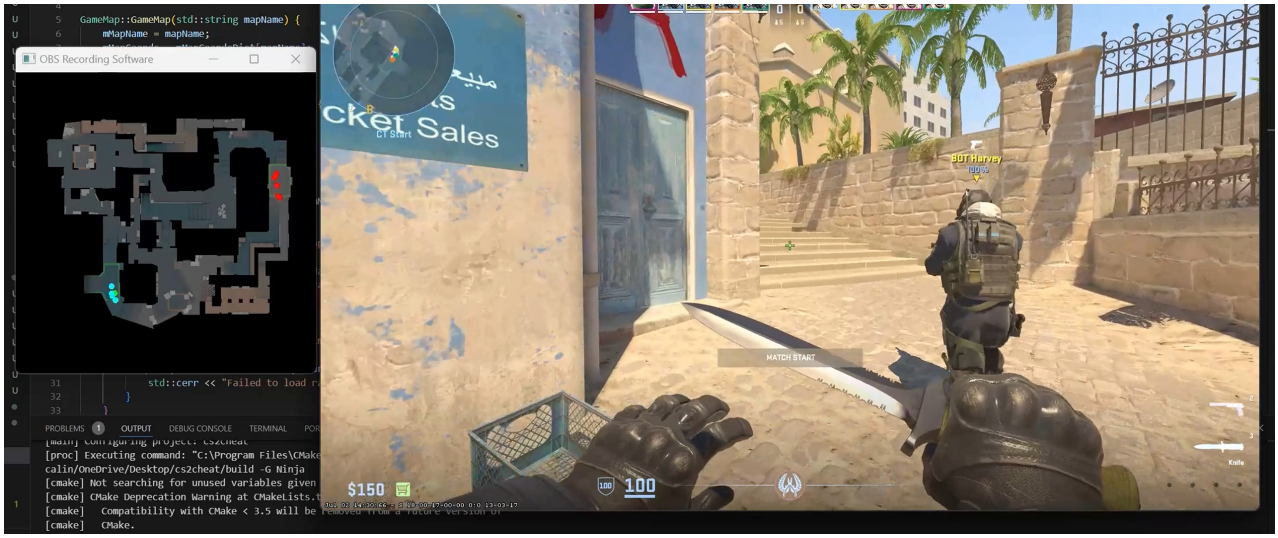

Figure 3: https://github.com/bordeanu05/cs2cheat

executing within highly privileged environments or utilizing standalone external hardware vectors. By exploiting low-level OS/Kernel interactions and dedicated hardware interfaces, these tools minimize or entirely eliminate their local software footprint, rendering conventional user-mode memory scanners and API hooks ineffective.

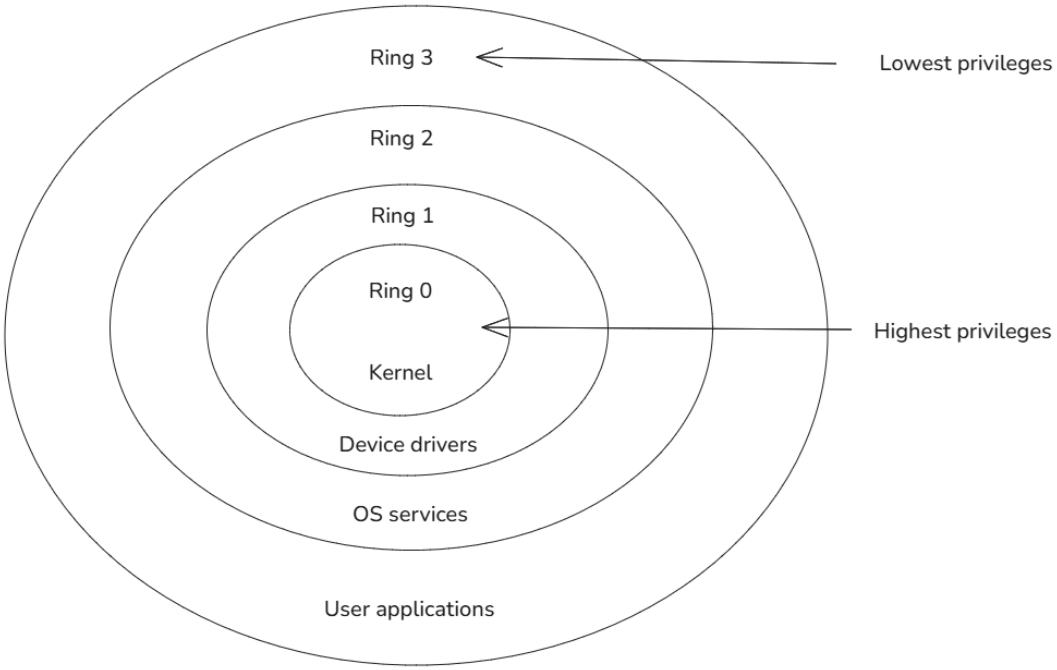

Figure 4: https://www.tutorialspoint.com/article/protection-ring

- Kernel-Level Cheats and Kernel interaction: Kernel-level cheats operate inside the most privileged layer of the computer's processor architecture, known as Kernel Mode (Ring 0). While regular video games and applications are restricted to User Mode (Ring 3) where they have limited permissions, software running in Ring 0 has absolute control over the entire system's memory

and hardware. Because modern client-side anti-cheat systems also run at this kernel level to monitor the game, cheat developers must find ways to gain equal authority to hide their presence and subvert security controls — a dynamic directly reflected in the privilege ring architecture shown in Figure 4, where both kernel-level cheats and anti-cheats compete for the same Ring 0 authority, leaving user-mode applications in Ring 3 with no visibility over either.

This advanced interaction with the operating system occurs through two primary mechanisms:

  - System Penetration via BYOVD (BYOVD): Modern operating systems like Windows require all kernel-level software to be digitally signed and approved by Microsoft. To bypass this security check, cheat developers use a technique called Bring Your Own Vulnerable Driver (BYOVD). They take an old, legitimately signed utility driver (from a trusted company) that contains a known security flaw. The cheat loads this trusted driver into the system and exploits its vulnerability to inject unsigned, malicious cheat code directly into the secure kernel memory space.
  - Direct System Manipulation: Once established inside Ring 0, the cheat can modify the operating system's internal tracking structures. Using a method known as DKOM (DKOM), the cheat can erase its own program from the active process list or alter the system's memory maps. This disguises the cheat's memory footprint, making it look like a harmless, native operating system component.

- External Cheats and Hardware Interfaces: The most modern escalation in this technical conflict involves external and hardware-assisted cheats, which decouple the cheat execution environment from the host operating system running the game client. This paradigm relies on physical hardware interfaces, most notably Direct Memory Access (DMA) via specialized hardware expansion cards installed directly into the host machine's PCIe (PCIs) slots. The DMA card physically reads and writes to the system's volatile RAM (RAM) directly through the hardware controller, completely bypassing the CPU (CPU) and any software-level security controls enforced by the operating system kernel. This raw memory stream is transmitted via a physical interface to a secondary, completely isolated computer. This external machine parses the game state asynchronously,calculates visual overlays or aim vectors, and can send synthetic peripheral inputs back to the host via specialized hardware input spoofers or modified mouse firmware. Because no malicious code executes, modifies memory, or hooks processes on the primary gaming machine, hardware-assisted vectors leave zero software footprint for traditional detection engines to scan — Figure 5 details this infrastructure, showing how the PCIe card reads host RAM through the hardware controller and relays the raw memory stream to an isolated external machine responsible for processing and input spoofing.

1.2 Anti-Cheat Mechanisms

Faced with the escalating sophistication of cheating techniques described above, publishers have progressively layered three generations of countermeasures, each operating at a different level of system

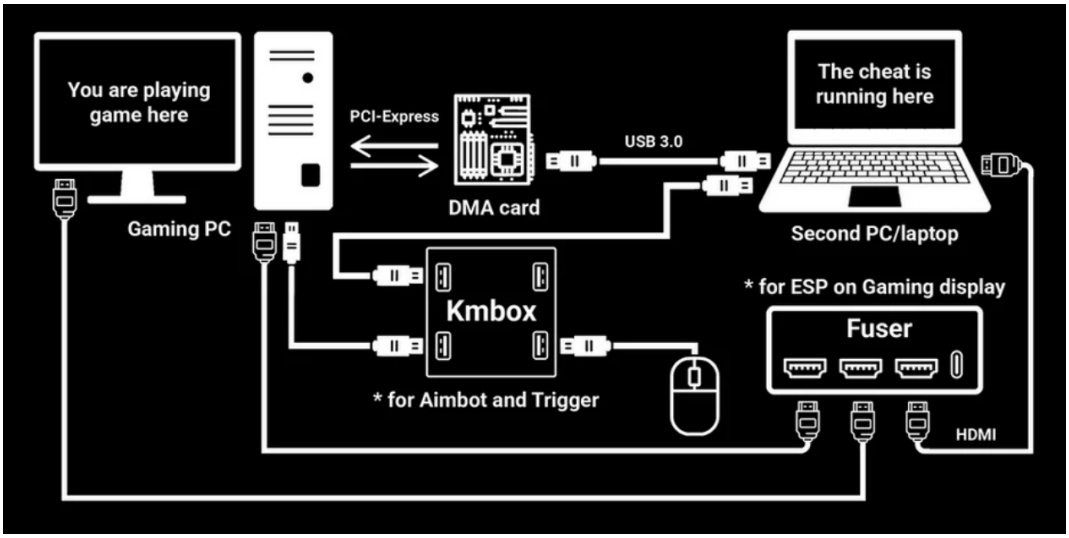

Figure 5: https://steams360.com/blogs/news/what-are-dma-cheats-and-how-does-it-work

privilege and each responding to the specific evasion vectors that the previous generation could not address.

1.2.1 Client-Side Anti-Cheats

Client-side anti-cheats operate within the same privilege boundary as the game itself — User Mode (Ring 3) — and rely primarily on two complementary techniques:

- Code Injection Detection: these systems monitor the process for signs of external code being loaded into the game's memory space, such as unauthorized DLL injections or suspicious calls to process-manipulation functions. By maintaining a whitelist of legitimate modules and flagging any unexpected addition to the process's loaded module list, this technique targets the injection-based automation cheats described in Section 1.1.1.
- Memory Scanning: the anti-cheat periodically inspects the game process's memory space, comparing observed byte patterns against a database of known cheat signatures, or applying heuristic checks for anomalous memory regions (unexpected executable permissions, unusually allocated pages). This is conceptually the same operation performed manually in the experimental section of Chapter 2, where Cheat Engine is used to locate a specific value within process memory.

Empirical benchmarking of eleven competitive multiplayer titles confirms that the most technically robust client-side systems enforce a layered combination of file integrity checks, code injection countermeasures, and process scanning, and that the price of commercially available cheats correlates strongly with the measured sturdiness of the anti-cheat software they must bypass [2].

Because these mechanisms remain confined to Ring 3, however, they share the same fundamental blind spot as the user-mode game client itself: any cheat operating at a higher privilege level, or entirely outside the host process, falls outside their observable scope — the limitation that motivated the shift toward kernel-level solutions.

1.2.2 Kernel-Level Anti-Cheats

Kernel-level anti-cheats close this visibility gap by operating at the same privilege level as the operating system itself — Ring 0 — mirroring the privilege escalation already adopted by advanced cheats (Section 1.1.3) and illustrated in Figure 4.

- Privileged Monitoring: operating in Ring 0 grants the anti-cheat driver direct access to the full process list, raw physical and virtual memory, and low-level system structures, independently of any cooperation from the monitored process. This allows continuous observation of system state that a Ring 3 component could never access unfiltered.
- Detection of Low-Level Cheats: this elevated vantage point specifically targets the evasion techniques described in Section 1.1.3 — detecting the unauthorized loading of vulnerable drivers exploited via BYOVD, and identifying inconsistencies introduced by DKOM manipulation, such as a process artificially removed from the kernel's active process list but still consuming system resources elsewhere.

Yet formal analysis of these systems against rootkit classification criteria reveals that at least two of the four most widely deployed kernel-level solutions — FACEIT Anti-Cheat and Vanguard — satisfy a majority of the behavioral metrics used to characterize rootkits, including boot-time execution, binary virtualisation, and continuous system data exfiltration [3]. This structural resemblance between the very tool meant to protect the system and the threat model it defends against is a central tension that Chapter 2 examines in detail, and that Appendix D develops further from a privacy and regulatory perspective.

1.2.3 Data-Driven Detection Methods

A third paradigm reframes the detection problem entirely by shifting the trust anchor away from the local machine and onto the game server, which — as the authoritative source of match state (Appendix C.1) — cannot be directly tampered with by the client.

- Shifting the Trust Anchor: rather than inspecting the player's machine for evidence of cheat software, this approach analyzes the gameplay data the server already collects, reducing dependence on client-side integrity and enabling a global, cross-match view of player statistics unavailable to any single local process.
- Player Behavior Modeling: this data is used to characterize performance patterns, consistency across matches, and statistical deviations from expected human behavior — for instance, reaction times, aim precision, or decision-making patterns that fall outside the range observed in legitimate play.

Recent work demonstrates the concrete feasibility of this shift: a transformer-based model trained on kill-event context windows extracted from 795 labelled CS2 matches achieves an accuracy of 89% and

an AUC of 0.93 on an unaugmented test set, while maintaining a low false-positive rate — a requirement particularly critical in competitive contexts where erroneous bans carry direct reputational costs [4].

This approach, however, is not without conceptual limitations. Behavioral models remain exposed to false positives, particularly when distinguishing between a genuinely skilled player and a cheater exhibiting similarly consistent performance — a tension commonly referred to as the high-skill bias. Legitimate but statistically unusual play patterns (an exceptional clutch, an outlier reaction time) can likewise be misclassified as suspicious. These limitations are precisely what motivate the design choices explored in depth in Chapter 3.

Chapter 2

Assessing Risks and Limitations of Kernel-Level Anti-Cheats using Windows-Kernel-Explorer

The programmers of tomorrow are the wizards of the future. You're going to look like you have magic powers compared to everybody else.

– Gabe NEWELL

2.1 Structural Limitations of Kernel-Level Anti-Cheats

2.1.1 Technological Arms Race

The deployment of kernel-level anti-cheat systems did not resolve the cheating problem: it escalated it. By elevating the detection layer to Ring 0, publishers effectively forced cheat developers to match that privilege level, triggering a continuous cycle of technical escalation in which each defensive advancement is systematically studied, reverse-engineered, and circumvented. This structural escalation is not merely theoretical: systematic grey-box testing of leading anti-cheat systems shows that even kernel-level solutions remain bypassed by a range of techniques, and that VAC (the system protecting CS2) ranks among the weakest evaluated, detecting only a narrow subset of the tested attack vectors [2].

This dynamic is structurally inherent to the paradigm. Once deployed, an anti-cheat driver enforces a static set of invariants (known cheat signatures, protected memory regions, hooked system calls, integrity checks) fixed at the moment of release. Cheat developers, operating in a well-funded and motivated ecosystem, treat each new version as a reverse-engineering target, often using the same kernel-level tooling described in Section 1.1.3 to map the detection surface before it can be updated.

The BYOVD technique (Section 1.1.3) is a direct product of this arms race: it emerged once anti-cheats began enforcing driver signature validation, prompting cheat developers to exploit the pre-existing trust chain of legitimately signed third-party drivers. Vendors responded with blocklists of known-vulnerable drivers — a measure Microsoft has since integrated at the OS (OS) level via its Vulnerable Driver Blocklist — while cheat developers shifted to newer, uncatalogued vulnerable drivers. Each iteration demands new engineering investment from the anti-cheat side, while the cheat ecosystem distributes evasion research across a community sharing tools openly. This asymmetry is documented in practice: cheat sellers maintain live detection-rate status pages, and a 37-day monitoring period shows at least one cheat remaining operational 100% of the time against kernel-protected titles, with median cheat uptime across all studied games above 85% [2].

This asymmetry reflects a deeper structural weakness: an anti-cheat update requires rigorous stability validation, since a defective kernel component can trigger system-wide crashes, corrupt system state, or introduce new vulnerabilities. The cheat ecosystem faces no such constraint: a new evasion technique can ship within days, while the corresponding patch may take weeks or months to safely deploy. The defender is therefore structurally slower than the attacker, regardless of resources invested, and each new game release, OS update, or hardware generation compounds this dependency: an unmaintained kernel component quickly becomes a liability, its Ring 0 presence introducing risk without corresponding detection benefit.

2.1.2 Limits Against New Forms of Cheating

Beyond the arms race, kernel-level anti-cheats are architecturally incapable of addressing cheat vectors that externalize execution outside the host OS entirely: a limitation rooted not in implementation quality but in the threat model itself.

Kernel-level anti-cheats assume the cheat leaves a detectable footprint within the host OS: unauthorized memory modification, suspicious process activity, unsigned driver loading, or anormal system function calls. This assumption breaks down entirely against DMA cheats (Section 1.1.3, Figure 5). A DMA cheat connects a dedicated PCIe card directly to the host's memory bus, reading process memory (including the decrypted Entity List) at the hardware level, bypassing the CPU and OS process model. Game state is transmitted to a physically separate machine for overlay rendering and aim computation, with synthetic inputs fed back via a hardware spoofer. Since no software executes on the host, no driver loads, no process is injected, and no local memory is modified, the kernel-level anti-cheat has no artifact to detect, since its Ring 0 monitoring simply never reaches the hardware interface involved.

This is a categorical defeat against a commercially available and increasingly prevalent class of cheats: a functional DMA setup (PCIe card, secondary machine, input spoofer) costs a few hundred dollars, well within reach of motivated cheaters in games where ranked progression carries economic or

reputational stakes. Software-side countermeasures (monitoring PCIe enumeration, measuring access-latency anomalies, detecting known DMA firmware signatures) share the same fundamental weakness as the broader arms race: they are signature-based, enumerable, and bypassable, since the attacker controls the hardware itself.

The architectural conclusion follows directly: kernel-level anti-cheat can only monitor the host OS processes. Any vector operating below that boundary (hardware memory access, external processing) falls entirely outside even the most privileged driver's detection perimeter.

2.1.3 Economic and Operational Costs

The structural limitations described above do not manifest in a vacuum: they translate into concrete and ongoing economic costs for publishers who choose to deploy kernel-level anti-cheat systems. These costs operate across three distinct dimensions that collectively challenge the long-term viability of the approach.

The first is driver maintenance. A kernel driver is not a product that can be shipped and forgotten: it must be continuously adapted to new OS versions, new hardware generations, and new security policies enforced by Microsoft's evolving Secure Boot and driver signing infrastructure. Each Windows update cycle introduces potential compatibility breaks that must be diagnosed and patched before they manifest as crashes or silent detection failures in the player base. This maintenance burden requires a dedicated engineering team with specialized kernel-level expertise — a skillset that is both scarce and expensive in the current software labor market.

The second is security incident management. As discussed in Appendix D.3, the mandatory installation of a Ring 0 component exposes the publisher to significant liability in the event of a security flaw. The historical record is unambiguous: the Capcom anti-cheat driver exploit allowed arbitrary unsigned code execution at kernel level on every machine where it was installed; the Genshin Impact anti-cheat driver was weaponized by a third party to deploy ransomware by exploiting its legitimate Ring 0 access. Each such incident triggers a crisis response cycle — emergency patching, public disclosure management, potential regulatory scrutiny, and, in severe cases, class-action litigation. The cost of a single major incident can exceed the cumulative engineering investment in the anti-cheat system itself.

The third is reputational and commercial impact. As detailed in Appendix D.2, a growing segment of the player base actively rejects kernel-level solutions, translating publisher anti-cheat decisions into measurable adoption losses. The exclusion of Linux and Steam Deck users (a platform whose market share has grown substantially with the mainstream success of Valve's handheld) from titles enforcing kernel drivers represents a quantifiable reduction in the addressable player pool. This exclusion is compounded by the architectural characteristics of the most aggressive solutions: always-on kernel drivers that boot independently of the game process and that, in the case of FACEIT Anti-Cheat, require users to disable Windows memory integrity protections and Hyper-V — a condition that exposes the system to a broader class of threats than those it is intended to prevent [3]. For free-to-play titles where player count directly drives monetization through network effects and cosmetic economies, this trade-off carries direct financial consequences that compound over time.

Taken together, these three cost vectors (maintenance, incident response, and adoption friction) establish that kernel-level anti-cheat is not simply a technical solution with technical limitations, but a strategic commitment with sustained organizational costs. The central question for publishers is therefore whether the detection effectiveness gained justifies this investment, particularly given the categorical limitations against hardware-assisted cheating vectors identified in Section 2.1.2. It is precisely this question that motivates the server-side behavioral approach explored in Chapter 3.

2.2 Experimental Section: Illustrating Potential Intrusiveness

2.2.1 Experimental Objective

- Illustrate the level of access of a privileged component
- Demonstrate security and privacy implications
- Headstart thanks to https://github.com/AxtMueller/Windows-Kernel-Explorer –> PROBLEM: ONLY SUPPORTED UNTIL WINDOWS 10. WHAT DO DO INSTEAD? I SEARCHED FOR ALTERNATIVES (READ RAM CONTENT ON LINUX, OTHER TOOLS TO READ RAM CONTENT ON WINDOWS) BUT COULDN'T GET ANYTHING TO WORK ON LINUX NOR WINDOWS. I COULD SHOW HOW CHEAT ENGINE CAN FIND A VARIABLE LOCATION IN RAM FROM USING SUCCESIVE VALUES TO PINPOINT THE EXACT LOCATION OF THE VARIABLE IN RAM (MEANING YOU CAN THEN READ IT OR MODIFY IT) BUT WOULD THAT BE REALLY USEFULL IN THE CONTENT OF THIS THESIS?

2.2.2 Experimental Observations

- Access to sensitive system resources
- Privilege differences compared to regular software
- Observation of non-game-related data

2.2.3 Critical Analysis

- Real vs theoretical implications
- Link with user perception
- Discussion of effectiveness vs intrusiveness proportionality

Chapter 3

Server-Side Approaches: Development of a Gameplay Analytics Pipeline for Suspicion Scoring

We can't solve today's problems with the mentality that created them.

– Albert EINSTEIN

3.1 Pipeline Implementation

PROGRESSION :

La pipeline est fonctionnelle, mon problème se situe pour le moment du côté de rendre la simulation des profils aussi réaliste que possible, puisqu'il n'est pas envisageable de parser les démos du jeu pour obtenir des données réelles (ça demanderait quelques années et une équipe de développeurs payés).

Pour le moment, les profils sont encore trop "parfaits" et ne reflètent pas la réalité des joueurs humains. Il faut que je trouve un moyen de générer des profils plus diverses, qui permettront de mettre en avant l'intérêt de la notion des LSTM pour détecter les comportements anormaux sur le long terme (et donc séparer les experts des cheaters).

Les modifications que je suis en train d'apporter se situent dans la création des profils de joueurs et dans la manière dont ils sont évalués par la pipeline pour mettre en avant des résultats plus proches de ce que l'on peut observer dans la réalité et de ce qu'on peut trouver dans les sources de données externes (voir annexe 5).

L'objectif final est de séparer les casuals des experts et des cheaters, et de pouvoir détecter les comportements anormaux sur le long terme pour amener ces profils à une revue manuelle de joueurs expérimentés qui sauront faire la distinction grâce à leur expérience. Les résultats graphiques que j'obtiens pour le moment sont les suivants :

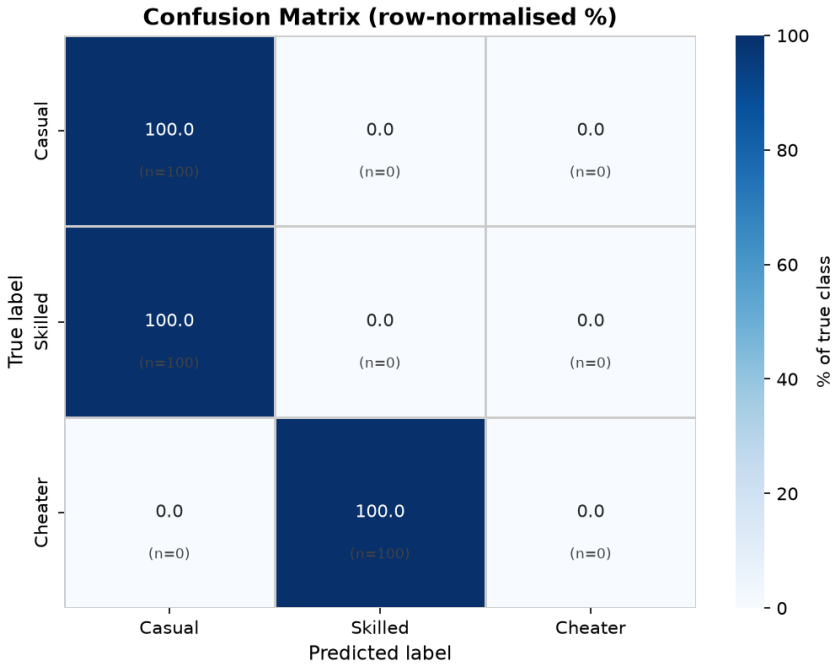

Figure 6: Confusion Matrix for Suspicion Scoring. Montre que l'algo de scoring n'arrive pas à différencier les profils de joueurs casual et expert. Il n'arrive pas également à différencier les profils de joueurs expert et cheater. Il est donc nécessaire de générer des profils plus réalistes pour que l'algo puisse apprendre à différencier les différents types de joueurs.

La repo est publique sur GitHub : https://github.com/selligre/episen-ing3-thesis-ac-server

3.1.1 Ingestion and Scoring Objectives

- Automating telemetry-driven suspicion scoring

Figure 7: Flag Frequency Distribution. Montre que l'algo de flagging se déclenche trop souvent pour les cheaters et pas assez pour les expert et les casuals. Je dois donc ajuster les seuils de déclenchement des flags pour avoir une représentation plus équilibrée et réaliste.

- Prioritization matrix for human review (Overwatch-style triage)

3.1.2 Algorithmic Architecture and Core Concepts

- Rule-Based Foundations: Establishing game engine limits, behavioral baselines, and mechanical consistency boundaries
- Hybrid Modeling Framework: Combining a classical deterministic heuristic algorithm (live run execution) with sequential Machine Learning concepts (Long Short-Term Memory - LSTM)
- Flag Weighting and Temporal Aggregation: Make individual mechanical anomalies accumulate into a player history score

3.1.3 Practical System Deployment

- Architecture of the analytics pipeline: https://github.com/selligre/thesis-cs2-ac-server (could use https://github.com/pnxenopoulos/awpy to help parsing data from demos)
- Event-driven design: Comparison of real-time telemetry parsing vs. asynchronous post-match processing

3.2 Dataset and Features

3.2.1 Telemetry Data Model

- Aim Vectors: TTD (TTD), pre-aim delta, and spray pattern deviation

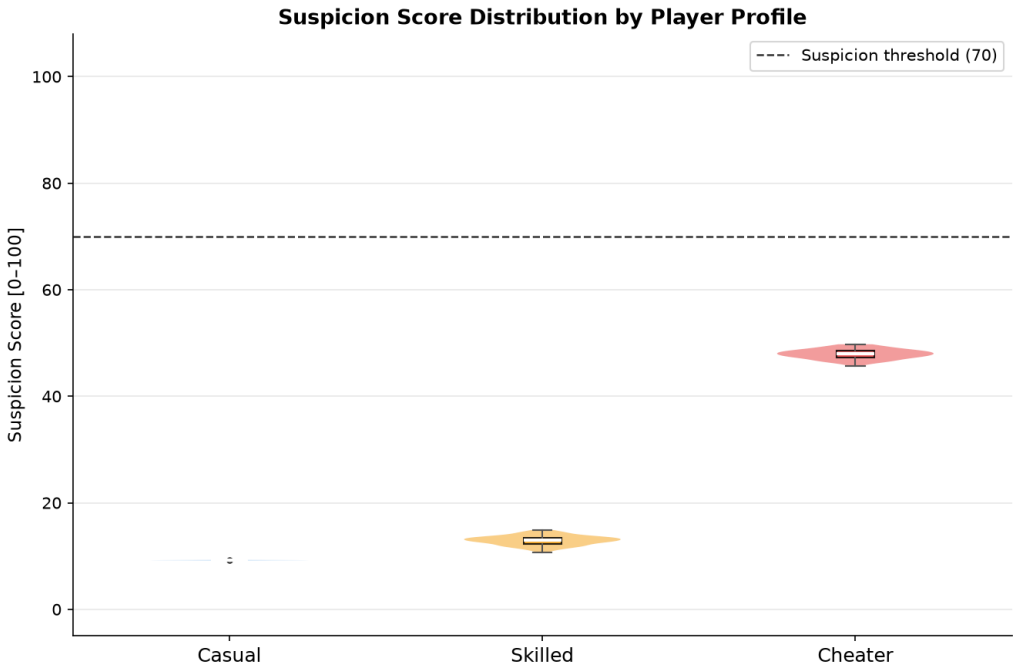

Figure 8: Score Distribution. Montre la distribution des scores attribués par l'algo de scoring. Le résultat est relativement satisfaisant pour le moment mais les scores sont globalement trop bas pour les experts et cheaters, donc les stats vont en moyenne être plus suspectes que le résultat actuel.

- Spatial and Positional Features: Cross-match positioning consistency, Opening Duels context, and Trading Efficiency windows
- Tactical and Utility Metrics: Grenade/HE efficiency (Average HE damage), Clutch performance variance, and overall Kill/Death ratio
- Social/Contextual Context: Premade lobby (Party) dependencies and cross-match behavioral

3.2.2 Data Sources and Profiling

- Synthetic telemetry generation: Simulating legitimate player profiles (varying from casual to highly-skilled) vs. simulated cheat profiles (aimbots, triggerbots, wallhacks and scripts)

3.2.3 Data Constraints and Limitations

- Limits of server-side logs
- Tick-rate limitations

3.3 Benchmark Execution and Raw Outputs

3.3.1 Experimental Protocol

- Setup of the controlled benchmarking environment

- Execution sequence: Batch processing of simulated profiles through the classical algorithm and the LSTM layer

3.3.2 Raw Evaluation Results

- Raw statistical outputs, distribution of suspicion scores and first results

3.4 Analysis and Performance Discussion

3.4.1 Net Detection Results

- Link between extreme data profiles and cheaters
- Robustness assessment: Evaluating the accuracy when mixing in high-skill profiles

3.4.2 Mitigating the High-Skill Bias

- Analyzing the conceptual limits of false positives
- Differentiating anomalous expert playstyles (exotic but legitimate behaviors) from software-assisted mechanics
- Comparative Analysis: Contrasting our suspicion-scoring pipeline with commercial performance platforms like Leetify.gg (How metrics designed for coaching can be repurposed for anti-cheat signaling)

3.4.3 Future Optimizations

- Build accurate data profiles faster
- Take into account external factors (number of cheaters in their friends list, age of the account, number of hours played related to the skill-level shown… things that experimented players learn to recognize but that the software fails to implement)

3.5 Comparison with Kernel-level approach

This capability gap is directly acknowledged in the literature: open-source behavioural detection trained exclusively on server-side kill telemetry has been shown to identify players subsequently confirmed as cheating via VAC ban, suggesting that hardware-level cheats — which leave no software footprint detectable by a kernel driver — nonetheless produce statistically anomalous gameplay signatures exploitable from the server [4].

- Effectiveness

- Intrusiveness
- Implications for the FPS / esport industry
- Player perception
- Human validation

Conclusion

The easiest way to stop piracy is not by putting antipiracy technology to work. It's by giving those people a service that's better than what they're receiving from the pirates.

– Gabe NEWELL

- Overall synthesis
- Limitations and future work
  - Lack of real data
  - Adaptation to further cheating methods

Ultimately, this thesis demonstrates that shifting toward server-side behavioral analysis resolves not only the critical security flaws of kernel-level access but also its severe socio-legal friction. By relocating the trust anchor from the user's local machine to the game server, developers can entirely bypass the ethical dilemmas associated with continuous Ring 0 surveillance. This paradigm shift aligns seamlessly with stringent global privacy regulations like the GDPR by eliminating the collection of non-game personal data. Furthermore, by replacing intrusive local monitoring with non-invasive gameplay analysis, this approach restores user trust and removes the barrier to adoption for privacy-conscious players, proving that safeguarding competitive fairness does not require compromising fundamental digital rights.

Bibliography

[1] Y. Kuzhii and Y. Furgala. "THE ANTICHEAT DEVELOPMENT IN THE COUNTER-STRIKE MAIN SERIES". In: *Electronics and Information Technologies*, vol. 26. 2024. DOI: 10.30970/eli.26.8. Available at URL: https://doi.org/10.30970/eli.26.8 (see page 1).

[2] Sam Collins et al. "Anti-Cheat: Attacks and the Effectiveness of Client-Side Defences". In: *Proceedings of the 2024 Workshop on Research on offensive and defensive techniques in the context of Man At The End (MATE) attacks*. ACM, pp. 30–43, 2024. DOI: 10.1145/3689934.3690816. Available at URL: https://doi.org/10.1145/3689934.3690816 (see pages 10, 13, 14).

[3] Christoph Dorner and Lukas Daniel Klausner. "If It Looks Like a Rootkit and Deceives Like a Rootkit: A Critical Examination of Kernel-Level Anti-Cheat Systems". In: *Proceedings of the 19th International Conference on Availability, Reliability and Security*. ACM, pp. 1–11, 2024. DOI: 10.1145/3664476.3670433. Available at URL: https://doi.org/10.1145/3664476.3670433 (see pages 11, 15, 35).

[4] Mille Mei Zhen Loo, Gert Luŝkov, and Paolo Burelli. "AntiCheatPT: A Transformer-Based Approach to Cheat Detection in Competitive Computer Games". In: *2025 IEEE Conference on Games (CoG)*. IEEE, pp. 1–4, 2025. DOI: 10.1109/CoG64752.2025.11114092. Available at URL: https://doi.org/10.1109/cog64752.2025.11114092 (see pages 12, 21).

Glossary

A

aimbot An automation cheat that systematically manipulates the view-angle variables stored within client process memory, enforcing automated frame-perfect vector corrections and target tracking that completely overrides the physical limitations of manual peripheral inputs. . . . **Page :** 1

C

cheat Software, hardware, or system modifications designed to provide an asymmetric, unfair competitive advantage to a player by subverting authoritative game engine mechanics, spatial information visibility, or user-input processing pipelines. . . . . **Page :** 2

client The local software instance of a video game running on the end-user's operating system, responsible for rendering local graphical vectors, capturing hardware peripheral inputs, and transmitting state packets asynchronously to a centralized authoritative network infrastructure. **Page :** 2

Counter-Strike 2 A tactical round-based first-person shooter developed by Valve, focusing on asymmetric objective match-ups between Terrorist and Counter-Terrorist factions, serving as the core empirical research and data evaluation framework for this thesis. . . . . **Page :** 1

cybersecurity The scientific discipline and operational practice of protecting digital systems, networks, programs, and endpoints from unauthorized exploitation, access, or subversion, encapsulating the architectural evaluation of privileged kernel drivers versus server-side privacy boundaries. . . . . **Page :** 2

D

driver A privileged kernel-mode software subsystem designed to abstract communications between the host operating system and physical hardware components, frequently weaponized by cheat developers via vulnerable third-party modules or deployed by anti-cheat systems for deep host visibility. . . . . **Page :** 3

E

esport Electronic Sports, a structured global ecosystem of professional competitive video gaming organized into high-stakes leagues, where competitive integrity, technical fairness, and robust anti-cheat validation are paramount to financial and structural stability. . . . . **Page :** 1

F

First-Person Shooter A highly competitive genre of action video games centered on weapon-based combat rendered from a direct first-person graphical perspective, placing critical reliance on rapid mechanical precision, situational awareness, and split-second cognitive responses. . **Page :** 1

K

kernel The foundational, core layer of an operating system operating at the highest execution privilege level (Ring 0), possessing unrestricted access to physical hardware channels, system volatile memory allocation, and low-level processor instructions. . . . . **Page :** 2

L

Leetify.gg An external third-party data analytics platform that parses match-demo data streams to compute complex gameplay performance analytics (such as aim tracking and trading position efficiency), serving as the core conceptual inspiration for the server-side behavioral pipeline developed in this work. . . . . **Page :** 21

M

Microsoft Microsoft Corporation, the developer of the Windows operating system, whose secure boot architectures, device driver digital signature mandates, and low-level API design define the modern technical interface boundaries for client-side security software. . . . . **Page :** 9

O

Overwatch A decentralized crowdsourced tribunal system historically implemented within Valve's ecosystem, allowing experienced, trusted human reviewers to audit anonymized game replay telemetry to validate or reject suspicious flags produced by anti-cheat systems. . . . **Page :** 19

R

radarhack An information cheat that extracts real-time physical spatial coordinates (X, Y, Z vectors) of hidden entities from volatile client memory space or intercepted network socket streams, projecting opponent positions onto an external clean secondary canvas or mini-map interface. **Page :** 6

rings The hierarchical privilege rings architecture implemented within modern central processing units (CPUs) to isolate executing software, ranging from the highly restricted User Mode (Ring 3) to the absolute system authority of Kernel Mode (Ring 0). . . . . **Page :** iv

S

script An execution sequence consisting of pre-compiled instructions or high-frequency input macros designed to automate complex, frame-dependent input timings, bypassing manual mechanical execution constraints such as perfect jump-throw synchronization or rapid spatial hops. **Page :** 4

server The central, authoritative computing node in a multiplayer network architecture that validates game state logic, enforces simulation rules, manages synchronization across all connected clients, and generates deterministic telemetry data streams. . . . . **Page :** 2

T

tick The elementary discrete discrete time-step execution unit of a multiplayer game server loop, defining the temporal frequency (measured in Hertz) at which the server processes client inputs, re-evaluates physical variables, and serializes state packets. . . . . **Page :** 5

V

Valve Valve Corporation, the American video game developer and digital distribution publisher responsible for creating the Source and Source 2 game engines, the Steam ecosystem, and the authoritative competitive Counter-Strike series. . . . . **Page :** 1

W

wallhack A category of information cheat that intercepts and alters client-side graphics rendering application programming interfaces (APIs) by disabling occlusion culling or depth-testing rules, granting an unauthorized real-time visual overlay of opponent models through solid opaque terrain geometry. . . . . **Page :** 1

Acronyms

API Application Programming Interface . . . . . **Page :** 6

BYOVD Bring Your Own Vulnerable Driver . . . . . **Page :** 9

CPU Central Processing Unit . . . . . **Page :** 9

DKOM Direct Kernel Object Manipulation . . . . . **Page :** 9

DMA Direct Memory Access . . . . . **Page :** 3

ESP Extra Sensory Perception . . . . . **Page :** 6

HE High Explosive . . . . . **Page :** 20

LAN Local Area Network . . . . . **Page :** 1

LSTM Long Short-Term Memory . . . . . **Page :** 3

OS Operating System . . . . . **Page :** 14

PCIe Peripheral Component Interconnect Express . . . . . **Page :** 9

RAM Random Access Memory . . . . . **Page :** 9

TTD Time To Damage . . . . . **Page :** 19

VAC Valve Anti-Cheat . . . . . **Page :** 1

Appendix A

Game Context and Rules

A.1 Competitive Format

Counter-Strike 2 (CS2) is a tactical, round-based first-person shooter played in 5-versus-5 matches between a Terrorist (T) and a Counter-Terrorist (CT) side. In the default competitive mode, a match consists of up to 24 rounds (12 per half before sides switch), each lasting a maximum of approximately 1 minute 55 seconds. A round is won either by completing an objective (planting and detonating the bomb for the T side, or defusing it / eliminating the opposing team for the CT side), by eliminating the entire opposing team, or by the timer running out.

Each player starts a match with a fixed amount of in-game currency and earns or loses money based on round outcomes, kills, and objective completions. This currency is spent during a "buy phase" at the start of each round on weapons, grenades (utility), and armor (Kevlar vest and helmet). This economic layer means that a player's loadout — and therefore their potential for damage output — varies from round to round, which is directly relevant to the telemetry features discussed in Chapter 3 (e.g. "premade lobby dependencies", utility efficiency).

A.2 Core Mechanics Relevant to Cheating

Several mechanical systems are central to understanding why the cheats described in Chapter 1 provide such a significant advantage:

- Recoil and spray patterns: each weapon follows a fixed, deterministic recoil pattern when fired continuously ("spraying"). Skilled players memorize and compensate for this pattern by moving their mouse in the opposite direction ("spray control"). An aimbot or recoil-compensation script can replicate this perfectly and instantly, which is the mechanism illustrated in Figure 2.
- Movement and "counter-strafing": CS2 applies an accuracy penalty while a player is moving. Precise, frame-perfect counter-strafing (briefly tapping the opposite movement key to cancel momentum before shooting) restores full accuracy. This is a mechanical skill ceiling that automation cheats can trivially exceed.

- Sound and information design ("fog of war"): as introduced in Section 1.1.2, large parts of player decision-making rely on incomplete information — footstep sounds, gunfire direction, utility usage, and map knowledge. The "mini-map" only shows teammates and recently-spotted enemies. This deliberate information asymmetry is the foundation that wallhacks and radarhacks subvert.
- Damage model and hit zones: unlike some shooters, CS2 does not apply a fixed "headshot multiplier" as a flat bonus; rather, each of the four hit zones on a player model (head, chest/arms, stomach, legs) carries its own damage multiplier applied to the weapon's base damage, with the head dealing roughly four times the damage of a torso hit, the stomach somewhat more than the torso, and the legs somewhat less. This zone-dependent damage model is precisely why aim assistance (aimbots) is so impactful: it does not merely guarantee a hit, it guarantees a hit on the highest-value zone (the head), turning encounters that would normally require multiple shots into immediate eliminations.

Appendix B

Why Cheating Breaks Game Balance: The Bullet Penetration Case

B.1 The Penetration Power Mechanic

In CS2, every weapon has a statistic generally referred to as "Penetration Power" (sometimes informally called "obstacle penetration"), which is distinct from "Armor Penetration" (the percentage of damage retained against a Kevlar vest, discussed separately in commercial weapon guides). Penetration Power governs how much damage a bullet retains after passing through an environmental surface — a wall, door, crate, or similar obstacle.

Crucially, this statistic is not expressed as a simple percentage and is not uniform across weapon categories. As a general rule:

- Pistols and submachine guns have the lowest penetration values and lose most of their damage when fired through cover, often dealing negligible or zero damage to a target behind a moderately thick wall.
- Assault rifles (AK-47, M4A4/M4A1-S, etc.) have a markedly higher penetration value, allowing them to retain a meaningful fraction of their damage through thin-to-medium obstacles such as wood, drywall, or thin metal.
- Sniper rifles (AWP, SSG 08, SCAR-20, G3SG1) have the highest penetration values in the game, making them the most effective weapons for shooting through cover.

The actual damage delivered after penetration depends on three combined factors: the weapon's Penetration Power, the material and thickness of the obstacle (thin drywall absorbs little damage; concrete or steel absorbs much more or blocks the shot entirely with the total number of surfaces a single bullet can penetrate capped at four), and the distance the bullet has already travelled before and after penetration with thin metal and drywall retaining 40–70% of damage while concrete or multi-layered walls can reduce damage by 70–90%, often rendering low-penetration weapons such as SMGs or pistols nearly useless.

1 2

B.2 Link to Information Cheats

This mechanic is the missing piece that explains why wallhacks (Section 1.1.2) represent such a disproportionate advantage, beyond simply "seeing through walls":

1. Legitimate uncertainty: an honest player knows, in principle, that bullets can pass through certain surfaces, and experienced players memorize a limited number of well-known "wallbang" spots on each map. However, they cannot know, in real time, the exact position of an unseen enemy behind that surface, nor precisely how much damage their shot will deal once combined factors (material, thickness, distance, weapon) are taken into account. Firing through a wall "blind" is therefore a probabilistic, high-risk action — a guess informed by sound cues, game sense, and map knowledge, not a guaranteed kill.

2. What a wallhack removes: a wallhack (Figure 3) eliminates this uncertainty entirely. The cheater sees the enemy's exact silhouette, position, and hit zone (head, chest, etc.) through the wall in real time. Combined with the penetration mechanic above, this transforms a probabilistic "blind" shot into a deterministic one: the cheater can wait until the visible target's head is aligned with a high-penetration weapon (e.g. an AWP or rifle) and a sufficiently thin surface, then fire with full confidence of landing a lethal headshot — something a legitimate player could never reliably replicate without the visual information.

3. Compounding effect with aim assistance: when combined with an aimbot, the penetration mechanic becomes almost irrelevant as a defensive layer for legitimate players, since the cheat can both locate the target through any surface and snap the crosshair to the highest-damage hit zone the instant a penetrable line of sight exists.

This illustrates concretely why the "fog of war" described in Section 1.1.2 is not a cosmetic feature but a core balancing mechanism: the penetration system was designed by Valve under the assumption that players cannot see what they are shooting at, and that the resulting uncertainty offsets the raw power of high-penetration weapons. Information cheats break this assumption without requiring any change to the weapon's underlying damage values, which is also why such cheats are extremely difficult to detect from server-side damage logs alone — the shot itself is mechanically "legal."

[1] https://counterstrike.fandom.com/wiki/Bullet_Penetration
[2] https://cs2pulse.com/wallbangs/

Appendix C

Client/Server Architecture and Network Communication

C.1 The Authoritative Server Model

CS2 follows a client-server architecture in which the server is authoritative over the game state. At a fixed frequency (the "tick rate"), the server receives input commands from each connected client (movement, view angles, button presses), simulates the resulting game state (player positions, weapon fire, damage, bomb state, etc.), and serializes a snapshot of the world state back to each client.

C.2 Snapshot Scope and Client Memory

Each client receives a snapshot covering all entities in the game world — including the positions, health values, and state of every player, regardless of their visibility to the receiving client. The client-side engine is responsible for rendering only what the local player should legitimately see; the underlying data for all entities is present in the client process's memory regardless.

After reception, the snapshot is decrypted and parsed into the Entity List — a data structure held in RAM containing plaintext 3D coordinates, health values, velocity vectors, and other attributes for every entity in the game. This in-memory representation is what the local engine uses to run prediction and rendering logic.

C.3 Network Channel Encryption

CS2 traffic is relayed through Valve's Steam Datagram Relay (SDR) infrastructure. The underlying networking library performs a Diffie-Hellman key exchange to establish an encrypted channel between client and server. Traffic is additionally relayed so that neither players' nor servers' IP addresses are exposed to each other. As a result, the snapshot payload is encrypted end-to-end at the transport layer.

C.4 Memory-Based Exploit Surface

Because the full world state is present in plaintext in the client process's RAM after decryption, it can be read by any component with access to that memory space. Radarhacks and wallhacks operate on this basis: a separate process reads the Entity List and uses the extracted coordinates to render enemy positions on a minimap or through geometry — data the client holds locally but would not normally display.

C.5 DMA Cheats

Direct Memory Access (DMA) cheats exploit the same condition via a hardware peripheral, typically connected via PCIe, that reads host memory from outside the operating system's process model. Because the access occurs at the hardware level, it does not appear as a software memory operation within the OS, making it transparent to both userland and kernel-level monitoring.

Appendix D

Socio-Legal, Privacy, and Community Dynamics of Kernel-Level Anti-Cheat Implementations

D.1 System Visibility Level and Perceived Intrusion

The architectural integration of anti-cheat software into the kernel layer shifts the boundaries of data access. Operating at Ring 0, a driver possesses the same execution privileges as the core operating system, effectively rendering the traditional memory protection boundaries of user-mode (Ring 3) obsolete.

- Theoretical Access to Non-Game-Related Data: Because kernel drivers have unrestricted access to the entire Virtual Address Space, they can theoretically read and monitor memory pages belonging to unrelated applications. This includes open web browsers containing active sessions, password managers, encryption keys, and local files. While publishers assert that their drivers only scan for cheat signatures, the technical capability to perform deep packet and memory inspection across the entire system remains a valid point of concern for privacy advocates. Empirical reverse-engineering of BattlEye's network behavior confirms this concern: the system continuously transmits to its backend infrastructure detailed information about all running processes, open handles, loaded modules, and game file sizes, regardless of whether the player is suspected of cheating [3].

- Continuous vs. Occasional Monitoring: A major shift occurred with the introduction of Riot Games' Vanguard (vgk.sys), which mandates an "always-on" approach. Unlike traditional anti-cheats that initialize alongside the game client and terminate upon closure, always-on kernel drivers boot during the system startup sequence. The industry justification is to prevent the pre-loading of cheats or hypervisors before the anti-cheat can initialize. However, this creates a state of continuous monitoring, leaving a privileged third-party component active even when the user is performing sensitive tasks unrelated to gaming.

- The Notion of Implicit Consent: End-User License Agreements (EULAs) operate on a "take-it-or-

leave-it" basis. Users are faced with a asymmetric choice: surrender Ring 0 access to their personal computer or lose access to the digital service entirely. In academic literature, this undermines the principle of freely given and informed consent, as the economic and social pressure to maintain connection with peer groups in modern video games forces users to accept invasive telemetric conditions.

D.2 Social Perception and Community Acceptance

The deployment of kernel-level anti-cheats has transformed a purely technical challenge into a highly polarized cultural debate within the global gaming community.

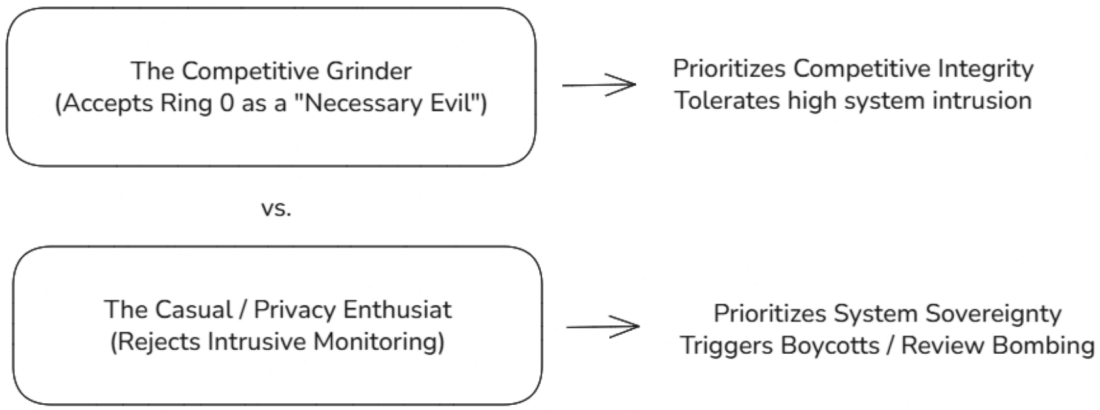

Figure 9: User Segmentation & Acceptance Divide, highlighting the contrasting perspectives of competitive and casual players regarding intrusive anti-cheat measures.

- Public Debates Around Intrusive Anti-Cheats: High-profile releases have frequently triggered severe community backlash. For example, the launch of Helldivers 2 faced significant criticism and review-bombing on platforms like Steam due to its implementation of nProtect GameGuard. Users frequently cite performance degradation, potential system instability, and the lack of transparency regarding what data is transmitted back to external servers as primary grievances.
- The Divide Between Competitive and Casual Players: The player base is structurally divided regarding acceptability, as summarized in Figure 6: competitive players tend to accept Ring 0 access as a necessary trade-off for integrity, while casual users and privacy advocates reject it as disproportionate, often responding with boycotts or review-bombing campaigns. Highly competitive players, particularly within esports ecosystems, often champion intrusive client-side measures, viewing them as a necessary tool to safeguard competitive integrity and financial stakes. Conversely, casual players or hardware enthusiasts view these measures as disproportionate. This divide is further amplified by ecosystem fragmentation, such as the complete exclusion of Linux and Steam Deck users from titles like League of Legends or Apex Legends due to the technical incompatibility of kernel drivers with translation layers like Proton.
- Impact on Game Adoption: This friction directly impacts the commercial viability of software. A growing segment of privacy-conscious consumers actively boycotts games utilizing kernel-

level solutions. For publishers, this creates an operational trade-off: mitigating financial losses caused by cheaters versus losing potential customers who refuse to compromise their system sovereignty.

D.3 Legal and Regulatory Questions

From a legal perspective, the deep monitoring required by client-side kernel components tests the boundaries of modern data protection frameworks and civil liability.

- GDPR Compliance: Proportionality and Data Minimization: Under Article 5(1)(c) of the General Data Protection Regulation (GDPR), personal data processing must be adequate, relevant, and limited to what is necessary in relation to the purposes for which they are processed (data minimization). Proving that an anti-cheat requires sweeping visibility over a user's entire operating system to catch cheaters represents a precarious legal position. If a server-side or behavioral approach can achieve similar outcomes without local system surveillance, the kernel-level approach may fail the core GDPR test of proportionality.
- Liability in Case of a Security Flaw: Standard EULAs explicitly disclaim all corporate liability for software-induced damages, hardware conflicts, or data breaches. However, from a consumer rights perspective, the mandatory installation of a software component capable of creating an exploitable system vulnerability (e.g., the historical Capcom driver exploit or the Genshin Impact anti-cheat exploit used to deploy ransomware) challenges these liability waivers. Should a major anti-cheat driver be weaponized on a global scale, publishers could face unprecedented class-action lawsuits concerning negligence in secure software design.
- Transparency and Terms of Service: Most publishers offer limited visibility into the telemetry data collected by their kernel components. The lack of open-source auditing or independent third-party verification means users must blindly trust corporate assertions. To align with modern transparency mandates, anti-cheat developers face growing pressure to provide detailed documentation, open APIs, or clear, granular logs showing precisely what memory segments are being analyzed and when data is being exfiltrated to the cloud.

Appendix E

Other appendixes

E.1 Cheaters in CS2

Community-driven data aggregation suggests that the actual prevalence of cheaters in CS2 matchmaking is lower than subjective perception implies — Figure 7 shows one such analysis, estimating a 6.6% chance of a cheater being present in a given game across approximately 3,000 matches at an average elo of 6,900.

E.2 Existing data aggregation tool

Third-party platforms already aggregate match telemetry into rich per-player behavioral profiles, as shown in Figure 8, demonstrating that demo-derived data is sufficiently granular to serve as a foundation for the suspicion-scoring pipeline developed in Chapter 3.

Abstract: Awesome abstract.

Keywords: Key, Words.

Figure 10: https://steamcommunity.com/app/730/discussions/0/4753074843849422337/

Figure 11: https://csst.at/profile/76561198261637028